

OMNIKON NATIONAL HACKATHON 2026 / FINAL BUILD

ChalkBox

A premium Classroom Teaching Engine for the classroom a teacher actually has.

PROBLEM	TEAM	PARTICIPANT
Omni_EdTech_7	Team HarshLabs	Harshawardhan Chitnis

Your classroom. Your plan. Powered by HarshLabs AI.

01 / EXECUTIVE SUMMARY

A complete classroom operating loop, not a text generator.

ChalkBox reduces lesson-planning friction for teachers who have limited time, materials, connectivity, and support. Its final loop is Plan - Structure - Teach - Assess - Reflect - Reuse.

The challenge

Teachers in under-resourced schools often prepare across multiple grades and subjects without dependable internet, printing, projectors, or ready-to-use materials. Generic AI output can add another editing burden when it ignores these constraints or stops at a paragraph of advice.

The product response

Need	ChalkBox response	Evidence in build
Speed	Structured plan from a bounded classroom brief	Constraint form, protected generation, prepared demo
Practicality	Exact-time typed teaching blocks	Common materials, alternatives and deterministic timing fit
Teacher control	Every output is editable and inspectable	Autosaving editor, quality panel, sources
Classroom continuity	Plan becomes a live teaching sequence	Progressive reveal, Present Mode, Quick Check 2.0, reflection
Trust	No secret in browser and no student profile	Edge Functions, RLS, privacy-safe data model
Resilience	Core demo and saved plans work without APIs	PWA, IndexedDB, explicit prepared flow

Core outcome

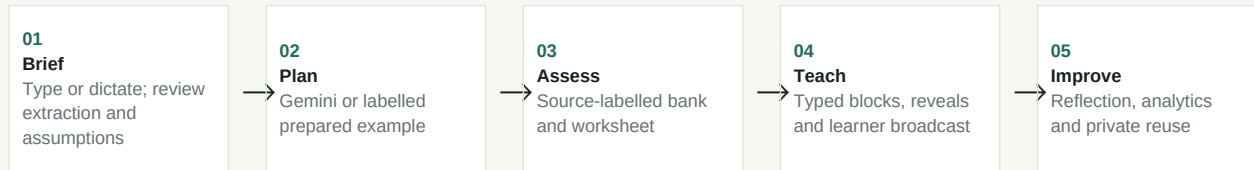
A teacher moves from a short brief to an exact-duration teaching sequence with code-native visuals, misconception handling, low-resource alternatives, anonymous response guidance, printable resources, and a reusable reflection loop.

Success is defined by the final working product: complete flows, strong UI/UX, transparent AI, offline behaviour, safe persistence, export, responsive design, and reproducible deployment.

02 / PRODUCT SCOPE

Plan, assess, teach, reflect, improve.

The main journey is intentionally linear for a first-time teacher and modular for a returning teacher.



Module	Implemented capabilities
Public and trust	Landing, about, privacy, 404, offline help, read-only share
Identity	Passwordless email, auth callback, onboarding, anonymous AI session
Teacher home	Next lesson, recent plans, outcome signal, assessment and classroom shortcuts
Brief and classroom	Voice/text extraction, review, mixed grade, custom board/subject, reusable context
Editor and history	Typed block editor, exact timing repair, Current vs Suggested AI review, checkpoints and restore
Assessment	Provenance, review state, add-to-lesson, worksheet and answer-key PDFs
Preview and share	Print, PDF, learner presentation and immutable expiring snapshots
Teach and reflect	Block/lesson timers, progressive reveal, Quick Check 2.0, class evidence and next action
Curriculum/community	Source-aware explorer, moderated immutable discovery, adaptation and reports
Insights/settings	Aggregate trends, offline queue, conflict choices, export and demo reset
Administration	Moderation queue, source audit, health, quota and validation telemetry

Explicit boundary

There are no student accounts, named learner records, automatic individual grading, mentor booking, payments, native apps, or silent AI fallbacks.

03 / JUDGE-READY DEMONSTRATION

Useful immediately after launch.

The deterministic demo proves the complete product even when provider setup, quota, or connectivity is unavailable.

Primary persona

Persona	Context	Seeded proof
Meera Patil	Grade 5-7 teacher at a fictional Zilla Parishad school in Pune district	4 plans, 2 classrooms, assessment bank, worksheet, versions, secure share, sessions, Quick Checks, reflection, publications and analytics

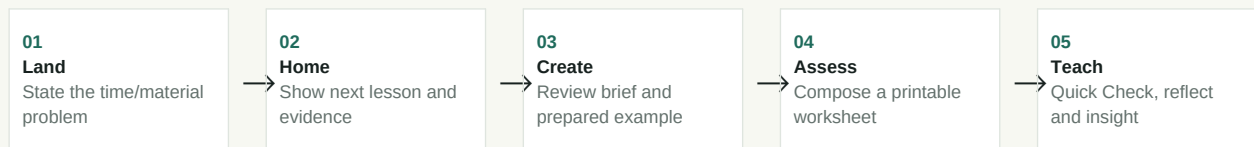
Prepared lesson records

Plan	Status	Classroom constraint	Demonstrated outcome
Photosynthesis: How Leaves Make Food	Ready	42 learners, bilingual, chalkboard, intermittent internet	Flagship 40-minute teaching engine, visual reveal, misconception, class pulse and exit ticket
The Water Cycle Around Us	Ready	38 learners, bilingual, no projector	Brief, history, assessment, teach/present, share, PDF and reflection
Fractions in the Local Market	Taught	Limited photocopies	Completed session, check-in and reflection analytics
A Story from Another View	Draft	42 learners, mixed reading levels	Return-to-draft and quality improvement state

Transparency controls

- Prepared plans carry a visible label in creation, editor, preview, share and export.
- The prepared button is a separate user action; it is never an automatic Gemini fallback.
- The fictional persona contains no real student or teacher personal information.
- Reset clears only ChalkBox browser tables and restores canonical fixtures.
- Approved community entries are immutable snapshots; a separate pending submission proves moderation.
- Analytics are calculated from fixture plans, sessions, reflections and aggregate Quick Checks.

Recommended five-minute path



Demo promise

No account, API key, database setup, or network connection is required after the PWA has loaded once.

04 / SYSTEM ARCHITECTURE

Simple at the edge, strong at the boundary.

A React PWA owns the experience; Supabase provides identity, persistence, policy, retrieval, and the secure server boundary.

Layer	Locked technology	Why
Frontend	React 19 + TypeScript + Vite	Fast SPA, strict contracts, efficient build and code splitting
UI	Tailwind CSS + native reusable components	Coherent responsive system without a heavy widget runtime
Routing	React Router	Public, protected, teacher, share and admin route boundaries
State	DomainProvider + Zustand + TanStack Query	Dexie-backed entities; UI/preferences separated from domain data
Forms	React Hook Form + Zod	Typed validation shared with AI input constraints
Offline	Dexie / IndexedDB + PWA	Reliable drafts, sessions and reflections on-device
Cloud	Supabase Auth, Postgres, Edge, pgvector	Free-first managed backend with RLS and vector search
AI	Gemini 3.7 Flash + Embedding 2	Strict classroom-block JSON and 768-dimensional retrieval
Output	React PDF + browser print	Private client-side document export
Quality	Vitest + Playwright + axe	Unit, component, lifecycle, mobile and accessibility tests
Delivery	Cloudflare Pages + GitHub Actions	Free CDN deployment, headers, SPA routes and CI

Data flow

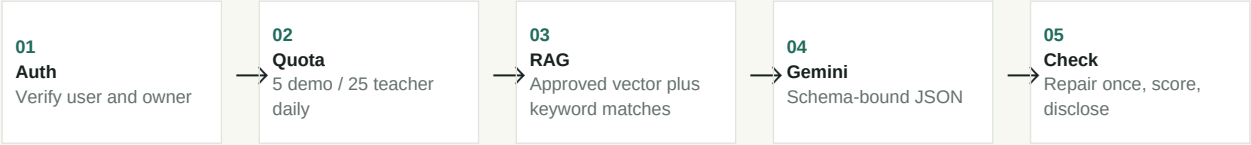


Resilience rule

A cloud failure cannot erase local work. Ordered mutations replay later; version mismatches offer Keep local, Keep cloud, or Duplicate both. Demo records never sync.

Generate structure. Preserve judgement.

Protected Edge Functions generate the same typed classroom contract used by the prepared flagship, then attach provenance from trusted retrieval code.



Control	Implementation	User-visible result
Secret isolation	Gemini key only in Supabase secrets	No key in Sources or network bundle
Input validation	Strict enums, lengths, counts and time bounds	Actionable 400 response
Source trust	Only approved hybrid matches with licence and attribution	Inspectable source cards
Output validation	JSON schema plus Zod	Malformed response never enters library
Repair	One bounded retry with validation issues	Reliable structure without retry loop
Review	AI questions enter as unreviewed variants	Teacher accepts before bank use
Quality	Objectives, exact timing, flow, grounding, offline and multigrade feasibility	Seven inspectable checks; exact means exact
Failure	Explicit provider/quota messages	Prepared content never masquerades as AI
Telemetry	ID, model, status, latency, retrieval, score	Operations without prompt contents

Copyright boundary

The participant-supplied NCERT Class VIII Science PDF is indexed only as 18 original concept summaries, page locators, ISBN, checksum and attribution. The repository and ZIP do not redistribute the book, figures, exercises or textbook prose.

Quick Brief rule-based parsing and prepared examples are labelled. Teacher review remains mandatory for current syllabus fit, facts, language, safety, accessibility, and the needs of the actual class.

06 / DATA, PRIVACY AND PERMISSIONS

No student profile is the strongest student privacy control.

The domain model stores teacher work and group-level classroom evidence only.

Entity	Persists	Access rule
Profile	Teacher identity and defaults	Self read/update; role column not self-editable
Lesson/version/share	Content, history and immutable publication snapshot	Owner CRUD; SHA-256 token hash stored; raw capability resolved only by RPC
Classroom	Aggregate count, resources, language and support context	Owner only; no student names
Question/worksheet	Provenance, review and immutable item snapshots	Owner or approved source policy
Teaching/Quick Check	Current step, notes and anonymous response counts	Owner only
Reflection	What worked, change, outcome, next action	Owner only
Publication/report	Submitted plan-version snapshot and moderation state	Approved public read; author/admin scoped writes
Curriculum	Source metadata, chunks and vectors	Approved read; admin write/index
Generation event	Operational metadata	Owner/admin read; service write

Security controls

- Postgres row-level security on every user/content/operation table.
- Anonymous users cannot SELECT the share table; a security-definer resolver checks only a cryptographic raw token, expiry and revocation.
- Owner ID checked both in RLS and inside the generation function.
- Admin role resolved with a security-definer helper and explicit admin gates.
- Content Security Policy permits only self, Supabase, Turnstile, local workers and data/blob assets.
- Turnstile integration is ready for passwordless and anonymous session abuse control.
- Remote writes use record versions; stale edits create an explicit conflict instead of overwriting.
- Environment files, provider keys, service keys, build output, browser binaries and local evidence are excluded from Git.

Teacher note boundary

The interface repeatedly says: capture a pattern, never a student name. There is no field or table for named student records.

07 / UX, ACCESSIBILITY AND OFFLINE

Calm enough for preparation. Clear enough for a live class.

The interface uses a paper-and-chalk visual language without sacrificing density, responsiveness, or keyboard clarity.

Design decision	Specification	Why it matters
Typography	System sans, strong 700-900 hierarchy	Fast loading and clear scanning
Palette	Moss green, warm paper, sun amber, coral alert	Distinct brand with restrained status use
Geometry	12-24 px radii, thin borders, low shadows	Friendly without visual noise
Layout	1440 px shell, 64/256 px navigation, mobile bottom bar	Efficient desktop and one-hand mobile use
Motion	Short transitions, reduced-motion support	Feedback without distraction
Accessibility	Landmarks, skip link, labels, focus rings, ARIA state	Keyboard and assistive-tech baseline
PDF scripts	Self-hosted Noto Sans Latin and Devanagari	Readable English, Hindi and bilingual exports
States	Loading, empty, offline, error, quota and missing record	No ambiguous blank screens
Offline	Precache app shell; persist plan/session/reflection	Teach through connectivity loss

Teach Mode priorities

- High-contrast typed block, grade target, purpose, exact lesson timer and per-block pacing timer.
- Private teacher cues and projection-safe learner content are structurally separated.
- Code-native visual models, misconception evidence, resource alternative and inclusive support remain one glance away.
- Pause, full screen, previous, next, skip, progressive reveal, lesson map and multigrade focus controls.
- Learner Present Mode receives a redacted broadcast object without teacher cues, notes, answers or corrective guidance.
- Read-aloud is available where the browser supports speech synthesis.
- Quick notes stored locally with a prominent no-student-name reminder.

Offline truth

AI generation and cloud sync require connectivity. Saved plans, editing, assessment resources, Teach Mode, reflection, print, PDF, and prepared demo remain available from the installed PWA.

08 / EVALUATION ALIGNMENT

Built against what is judged.

Every hackathon criterion maps to a concrete product and engineering proof point.

Criterion	ChalkBox evidence
Innovation and creativity	Quick Brief review, low-resource constraint engine, assessment-to-reflection loop, immutable trust model
Technical implementation	React PWA, IndexedDB, RLS Postgres, Edge Functions, pgvector, structured Gemini, PDF, CI
Problem solving	Starts from class size, materials, time, language and connectivity instead of idealised hardware
Scalability	Typed contracts, feature modules, vector index, owner indexes, rate limits, stateless Edge Functions
User experience	Responsive shell, voice assist, classroom profiles, autosave/history, Teach/Present, printable resources
Code quality	Strict TypeScript, Zod, repositories, shared domain package, lint, format, tests and documentation
Presentation and documentation	Seeded five-minute flow, report, demo guide, architecture, security and deployment docs

Competitive differentiation

Common lesson AI	ChalkBox
Prompt to text	Brief to validated typed classroom object
Stops after generation	Structure, reveal, assess, worksheet, teach, present, reflect and reuse
Assumes connected smart classroom	Offline alternative and real material constraints
Citation as model text	Server-derived provenance metadata
Demo breaks when API fails	Prepared and rule-based paths with explicit labels
Mutable public copy	Immutable expiring share and moderated publication snapshots
Opaque quality	Seven deterministic checks updated during editing
Analytics may imply student tracking	Teacher-only, group-level reflection signals

09 / VERIFICATION

Quality gates before the final URL.

The repository includes executable checks for contracts, logic, components, full journeys, mobile behaviour, and accessibility.

Gate	Coverage	Current package status
Fixture validation	Every seeded plan against shared schema	Script included
TypeScript	App, tests, scripts, configs and shared package	Pass
ESLint	TypeScript, React hooks and JSX accessibility	Pass with zero warnings
Unit/component	Contracts, exact timing, learner redaction, tokens, fixtures, store, sharing, moderation and queue	32 tests pass
Production build	Code splitting, PWA manifest and service worker	Pass
Browser lifecycle	Lesson, worksheet, community and mobile journeys	Playwright suite included; CI gate
Accessibility	WCAG A/AA serious and critical automated scan	axe suite included; CI gate

Never-break flows

- Enter the prepared demo without setup.
- Retain edited work after reload and recover it offline.
- Keep Gemini keys out of the browser and reject owner mismatch.
- Create immutable shares/publications and enforce expiry, revocation and moderation.
- Build learner and answer-key PDFs from provenance-labelled question snapshots.
- Preview, print, PDF export, public share and source disclosure.
- Teach all typed blocks, reveal progressively, capture a class signal, complete reflection and see derived analytics.
- Block guests from private routes and teachers from admin routes.
- Reset only the demo workspace to canonical seeded data.

Reproducibility

GitHub Actions installs with the frozen pnpm lockfile, uploads the production build, runs Playwright after fast gates, and preserves the browser report.

10 / FREE-FIRST DELIVERY

A complete build without spending INR 1.

The chosen services and limits keep the hackathon deployment inside free tiers while preserving a useful failure mode.

Service	Purpose	Cost control
Cloudflare Pages	Static PWA, CDN, TLS, headers and SPA routing	Free Pages project; immutable assets
Cloudflare Turnstile	CAPTCHA for auth/anonymous abuse	Free widget; optional until configured
Supabase	Auth, Postgres, RLS, Edge Functions, pgvector	Single free Mumbai project; usage alerts
Gemini	Lesson/actions JSON and embeddings	Free-tier models; stricter application quotas
GitHub	Public source, PR, Actions and build artefacts	Public repository and bounded CI
Browser storage	Offline plans and sessions	No service cost

Deployment sequence

1. Apply all four Supabase migrations and confirm RLS/RPC grants. 2. Set Edge Function secrets. 3. Deploy generation, AI action, indexing and health functions. 4. Index approved original and copyright-safe derived curriculum bundles. 5. Connect GitHub to Cloudflare Pages. 6. Add public variables and Auth callbacks. 7. Enable Turnstile after hostname verification. 8. Run incognito, offline, PDF, share, moderation and role checks.

Graceful quota behaviour

When a free AI quota is exhausted, the teacher can still open, edit, teach, duplicate, reflect, export, share saved plans, and use the prepared demo.

Definition of done

- Source, production build, migrations, functions, tests, CI, documentation, report and ZIP are present.
- Prepared demo completes the primary journey from a clean browser.
- Cloud features require only public browser variables and server-side secrets.
- Integration status distinguishes locally verified features from unconfigured hosted services.
- Final hosted verification covers desktop, mobile, offline, auth, live AI/RAG, PDF, share, moderation and roles.

11 / CLOSING

Give every teacher a better starting point.

ChalkBox does not try to replace the judgement, relationships, or local knowledge of a teacher. It removes avoidable preparation friction and makes the remaining decisions easier to inspect.

A useful lesson-planning system should not assume a projector, perfect internet, unlimited paper, one reading level, or an evening of free time.

Build

A complete, responsive, installable teacher classroom OS with Quick Brief, typed teaching engine, assessment, Teach/Present, reflection, secure sharing, moderation and a deterministic flagship path.

Innovate

Constraint-aware structured generation, hybrid attributable retrieval, immutable trust boundaries and the plan-to-reflection evidence loop.

Impact

More teacher time focused on learner thinking, with practical support that still works when resources do not.

Project	Team	Repository
ChalkBox	Team HarshLabs / Harshawardhan Chitnis	github.com/harshawardhanchitnis/Omnikon_Hackathon_Harshawardhan_Chitnis

Your classroom. Your plan. Powered by HarshLabs AI.